Building PowerToys Command Palette Extensions

A Complete Developer Guide

Subtitle: Step-by-step from environment setup to Gallery publishing

Version: v2.0 Updated for PowerToys 0.90+ and the Command Palette SDK (2025 / 2026)

Document Date: June 14, 2026

Audience: Intermediate C# Developers on Windows

Platform: Windows 10 / Windows 11 | .NET 8.0+ | Visual Studio 2022 / 2026

Who This Guide Is For

This guide is written for intermediate C# developers who are comfortable with object-oriented programming, Visual Studio, and the NuGet package ecosystem. You do not need prior experience with WinUI, COM, or Windows packaging — all required concepts are introduced step-by-step. If you have built a class library or a simple desktop app in C# before, you have everything you need to follow along. Familiarity with `async/await`, interfaces, and basic Windows concepts (Registry, file system, Developer Mode) is assumed but not strictly required.

What You Will Build

Throughout this guide you will construct a fully functional Command Palette extension called **QuickLinks**. QuickLinks lets users maintain a personal list of named URLs and open any of them instantly from the Command Palette — no browser address bar, no bookmarks toolbar, no alt-tabbing. Users invoke the extension, see a scrollable list of their links (e.g., "Jira Board," "Azure Portal," "Team Wiki"), and press **Enter** to launch the URL in their default browser. The guide then shows how to add a **Form Page** for adding new links at runtime, a **Markdown help page**, and how to persist settings between sessions. By the end you will

package the extension as an MSIX, publish it to WinGet, and list it in the Command Palette Gallery.

Table of Contents

Part 1 — Understanding the Command Palette Extension Platform

1.1 What Is Command Palette?

Command Palette is Microsoft's next-generation launcher for Windows, introduced as part of PowerToys and designed to eventually supersede the earlier PowerToys Run experience. Where PowerToys Run was a plugin-based search bar, Command Palette is an **extensibility-first, accessibility-first, full-UI platform** — built on top of WinUI 3 and the Windows App SDK — that gives developers the ability to embed rich interactive experiences (lists, forms, markdown views, grids) directly inside the palette without shipping a separate application window.

From the user's perspective, Command Palette is a single keyboard shortcut away (default: **Win+Alt+Space**). From the developer's perspective, it is a host process with a well-defined COM-based extension protocol. Your extension runs in its own process, communicates with the host over COM, and renders its UI entirely through built-in SDK page types. Users never leave the palette surface to interact with your extension.

Key design goals of Command Palette include:

- **Blazing-fast startup** — the palette appears in milliseconds; extensions load on-demand.
- **Accessibility-first** — full keyboard navigation, Narrator/screen-reader support, high-contrast themes.
- **Extensibility-first** — every surface in the palette is exposed through the same SDK that Microsoft uses internally.
- **No separate window required** — your extension's UI lives entirely within the Command Palette overlay.
- **.NET native** — extensions are written in C# targeting .NET 8+, with full access to the NuGet ecosystem.

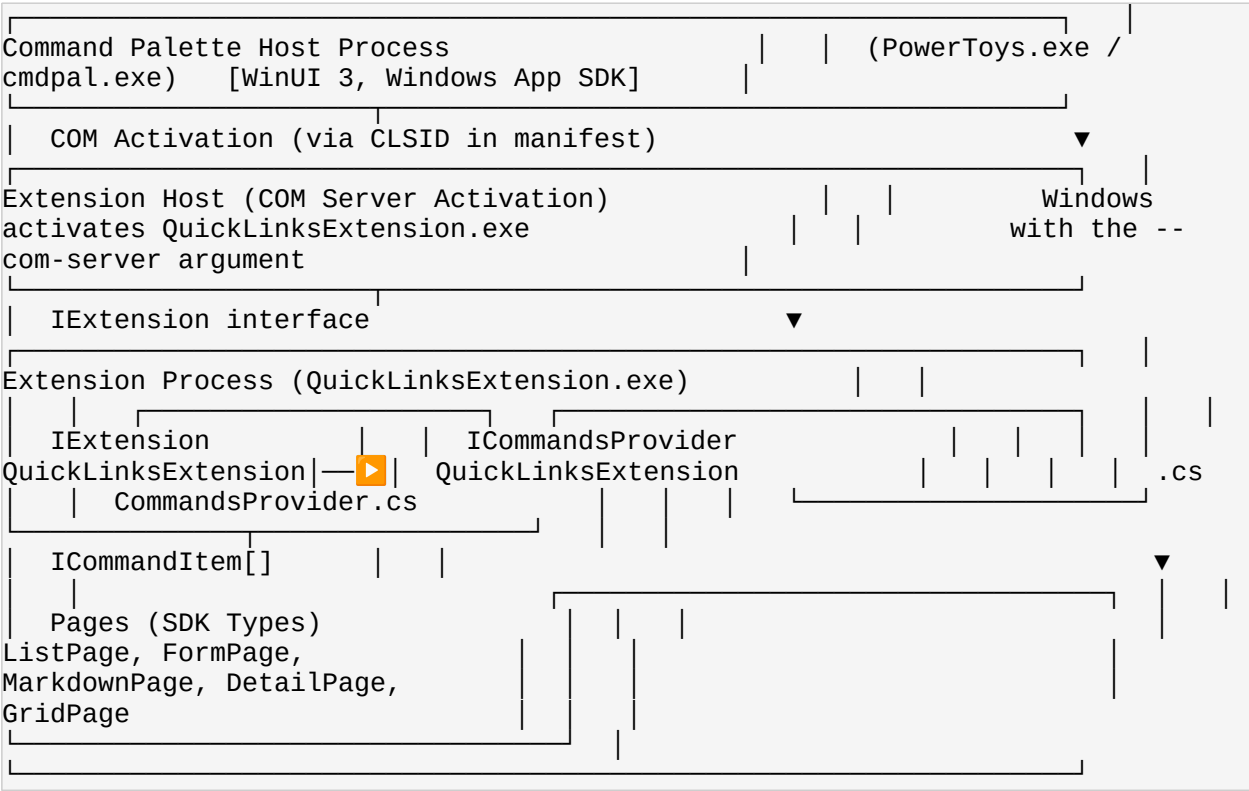
1.2 Extension Architecture Overview

Command Palette uses a **COM-based out-of-process (OOP) extension model**. This means your extension runs in a separate process from the Command Palette host. The host activates your extension through COM using a registered CLSID (Class Identifier), calls into your implementation of the `IExtension` interface, and then queries it for an `ICommandsProvider` that returns the commands your extension exposes.

This architecture has important implications:

- Your extension crashing does **not** crash the Command Palette host.
- Your extension runs with its own process memory — no shared state with other extensions.
- COM activation is handled automatically by the Windows Runtime; you register an executable as a COM server in your `Package.appxmanifest`.

The high-level communication flow is shown in the ASCII diagram below:



Key SDK types and interfaces:

- **IExtension** — The root interface your extension class must implement. The host calls `GetProvider<T>()` on it to retrieve capabilities (e.g., `ICommandsProvider`).
- **ICommandsProvider** — Returns the top-level array of `ICommandItem` objects that appear in the palette when a user searches for your extension.
- **ICommandItem** — Represents a single command entry with a name, icon, and an associated page or invokable action.
- **IListItem** — A single row in a `ListPage`, with title, subtitle, icon, and an `IInvokableCommand`.
- **IInvokableCommand** — An action that is called when the user presses Enter on a list item or command.
- **Page types** — Prebuilt UI containers: `ListPage`, `FormPage`, `MarkdownPage`, `DetailPage`, `GridPage`.

1.3 Page Types Reference

Page Type	Description	Best Used For
ListPage	Displays a scrollable, searchable list of `IListItem` rows. Each item has a title, optional subtitle, icon, and an invokable command. Supports keyboard navigation and incremental filtering.	Navigation menus, search results, file pickers, link launchers, recent items. The most common page type.
FormPage	Renders a structured form with labeled input fields (text, dropdown, checkbox, etc.) and a submit button. Handles validation and submission through the SDK.	Creating or editing records, configuration dialogs, input collection (e.g., add a new quick link).
MarkdownPage	Renders a string of Markdown as formatted text inside the palette.	Help pages, changelogs, rich text display, README-style content,

Page Type	Description	Best Used For
	Supports headings, lists, bold/italic, links, and code blocks.	documentation viewer.
DetailPage	Shows a single-item detail view with a hero title, optional image, and a set of labeled property rows. Similar to a "property sheet."	Displaying metadata about a single item (e.g., a file, a contact, a calendar event).
GridPage	Presents items as a grid of tiles with icons and labels. Supports keyboard navigation across rows and columns.	App launchers, emoji pickers, icon galleries, category browsers.

1.4 SDK NuGet Packages

The Command Palette extension SDK is distributed as two primary NuGet packages. Both are available on **nuget.org** and are automatically referenced when you scaffold a project using the built-in Create Extension command.

Package Name	Description	NuGet.org
`Microsoft.CommandPalette.Extensions`	The **core SDK**. Defines all interfaces: `IExtension`, `ICommandsProvider`, `ICommandItem`, `IListItem`, `IInvokableCommand`, all page interfaces, and `IExtensionSettings`. This is the minimum dependency for any extension.	nuget.org/packages/Microsoft.CommandPalette.Extensions
`Microsoft.CommandPalette.Extensions.Toolkit`	A **helper utility layer** on top of the core SDK. Provides concrete base classes — `ListPage`, `FormPage`, `MarkdownPage`, `ListItem`, `InvokableCommand`, `IconHelpers`, `OpenUrlCommand`, `CopyTextCommand` — that	nuget.org/packages/Microsoft.CommandPalette.Extensions.Toolkit

Package Name	Description	NuGet.org
	dramatically reduce the boilerplate required to build extensions. Strongly recommended for all new projects.	

> 💡 **Tip**
>
> Always use **Central Package Management** (the scaffolded `Directory.Packages.props` file) to keep package versions consistent across a multi-project solution. Never specify version numbers directly in individual `.csproj` files if the scaffolded solution structure is in place.

Part 2 — Setting Up the Development Environment

2.1 Prerequisites Checklist

Item	Minimum Version	Notes
Windows OS	Windows 10 Build 17134 (1803)	Windows 11 is strongly recommended for the best WinUI experience. Windows 10 is supported but some SDK features may differ.
Visual Studio	2022 (v17.4+) or Visual Studio 2026	Community edition is free and fully supported. Enterprise/Professional also work. VS Code is *not* supported for MSIX packaging.

Item	Minimum Version	Notes
.NET SDK	8.0	Extensions target `net8.0-windows10.0.22621.0` or later. Install via Visual Studio installer or from dotnet.microsoft.com.
Windows App SDK (WinUI workload)	1.7+	Required for MSIX packaging and WinUI 3 runtime. Installed via the "Windows application development" workload in VS Installer.
Windows 10 SDK	10.0.22621.0	Minimum target SDK. Install via VS Installer → Individual Components.
Windows 11 SDK	10.0.26100.0	Required to build against the latest Command Palette APIs. Install via VS Installer → Individual Components.
PowerToys	v0.90+	Must have Command Palette feature enabled in PowerToys Settings. Available from Microsoft Store or WinGet.
Developer Mode	Enabled	Required for side-loading (deploying) packaged apps without the Store. See Section 2.3.
Long Paths	Enabled	NuGet restore paths can exceed 260 characters. Must be enabled in Windows Registry or Group Policy. See Section 2.4.

2.2 Installing Visual Studio with Required Workloads

1. Open the **Visual Studio Installer** (search "Visual Studio Installer" in the Start menu). If Visual Studio is already installed, click **Modify** next to your installation.

2. On the **Workloads** tab, locate and check the following three workloads:
    - **Desktop development with C++** — provides MSVC build tools needed for Windows packaging.
    - **.NET desktop development** — provides the .NET 8 SDK and WPF/WinForms tooling.
    - **Windows application development (WinUI)** — provides the Windows App SDK, WinUI 3 designer, and MSIX packaging tools.
3. Switch to the **Individual Components** tab. In the search box, search for and check:
    - **Windows 10 SDK (10.0.22621.0)**
    - **Windows 11 SDK (10.0.26100.0)**
4. Click **Modify** in the lower-right corner. Allow the installer to download and apply changes. This may take 10–30 minutes depending on your internet connection.
5. Once complete, launch Visual Studio and verify the installation: **Help → About Microsoft Visual Studio** — confirm the .NET SDK version appears in the list.

[Screenshot: Visual Studio Installer — Workloads Tab]

The Visual Studio Installer Workloads tab is shown with three workloads highlighted in blue: "Desktop development with C++", ".NET desktop development", and "Windows application development (WinUI)". Each has a checkmark. The bottom-right "Modify" button is visible.

📝 **Note**

If you are using Visual Studio 2026, the "Windows application development" workload may be labeled slightly differently (e.g., "WinUI / Windows App SDK

development"). The required components are the same. Always verify the Windows App SDK runtime is included.

2.3 Enabling Windows Developer Mode

Developer Mode allows you to deploy (side-load) MSIX-packaged apps to your local machine without going through the Microsoft Store. Without it, Visual Studio's Deploy command will fail with a permissions error.

To enable Developer Mode:

Navigate to: **Settings → System → For developers → Developer Mode → toggle On**

On Windows 11: **Settings → Privacy & security → For developers → Developer Mode → On**

Windows will display a confirmation dialog warning you that installing apps from outside the Store carries risks. Acknowledge the dialog. The setting takes effect immediately — no restart required.

> ⚠️ **Important**
>
> Developer Mode is a per-machine setting, not per-user. If you are working on a managed corporate device, your IT policy may prevent enabling it. In that case, contact your administrator or use a personal/VM environment for extension development.

2.4 Enabling Long Paths

The .NET and NuGet restore pipelines can generate paths exceeding 260 characters (the historical Windows limit). Without Long Paths enabled, builds will fail with cryptic "path not found" errors.

Method A — Registry Editor

6. Open **Registry Editor** (`regedit.exe`) as Administrator.
7. Navigate to: `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem`
8. Double-click `LongPathsEnabled` and set the value to `1`.
9. Click **OK** and close Registry Editor. Restart required for changes to take effect.

Method B — PowerShell (Run as Administrator)

```
# Run PowerShell as Administrator, then execute: Set-ItemProperty `     -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem' `     -Name 'LongPathsEnabled' `     -Value 1  Write-Host "Long Paths enabled. Please restart your machine."
```

> 💡 **Tip**
>
> On Windows 11 with Group Policy enabled, you can also enable Long Paths via: **Local Group Policy Editor → Computer Configuration → Administrative Templates → System → Filesystem → Enable Win32 long paths → Enabled**.

2.5 Installing PowerToys

PowerToys must be installed and the Command Palette feature enabled for you to test your extension locally.

Option A — Microsoft Store

Search "Microsoft PowerToys" in the Microsoft Store app and click **Install**.

Option B — WinGet (Recommended)

```
winget install Microsoft.PowerToys --source winget
```

After installation, open PowerToys Settings and enable Command Palette:

Navigate to: **PowerToys Settings → Command Palette → Enable Command Palette → toggle On**

> **[Screenshot: PowerToys Settings — Command Palette Toggle]**
>
> *The PowerToys Settings window is open. The left sidebar shows "Command Palette" selected. The main panel shows the "Enable Command Palette" toggle switched to the On/blue position. The activation shortcut field shows "Win+Alt+Space".*

2.6 Automated Setup (Optional)

If you are building extensions that you intend to contribute back to the PowerToys open-source repository, or if you want a fully scripted environment setup, the PowerToys repository includes a setup script that handles all prerequisites automatically.

```
# Clone the PowerToys repository first: git clone
https://github.com/microsoft/PowerToys.git cd PowerToys  # Run the setup
script as Administrator: .\tools\build\setup-dev-environment.ps1
```

The script performs the following actions automatically:

- Enables Long Paths in the Registry.
- Enables Developer Mode in Windows Settings.
- Verifies and installs required Visual Studio workloads and individual components.
- Initializes all git submodules (the PowerToys repo uses submodules for several dependencies).

- Restores NuGet packages.

> 📝 **Note**
>
> The automated setup script is intended for PowerToys contributors who build the entire PowerToys solution. For standalone extension development, manual setup (Sections 2.1–2.5) is sufficient and faster.

Part 3 — Scaffolding a New Extension Project

3.1 Using the Built-in "Create Extension" Command (Recommended)

The fastest way to start a new Command Palette extension is to use the **built-in scaffolding command** that ships with Command Palette itself. This generates a complete, ready-to-build Visual Studio solution with all the correct project references, manifest entries, publish profiles, and directory structure already in place.

Field	Value for This Guide	Rules
Extension Name	`QuickLinksExtension`	No spaces. Must be a valid C# class name. Must begin with an uppercase letter. This is used as the root namespace and file name prefix.
Extension Display Name	`Quick Links`	The human-readable name shown in Command Palette. Spaces allowed.
Output Path	`C:\Dev\`	The folder where the

Field	Value for This Guide	Rules
	`QuickLinksExtension`	solution will be generated. Must exist or will be created.

> **[Screenshot: Command Palette — "Create a new extension" Form Filled Out]**
>
> *The Command Palette overlay is open showing a Form Page titled "Create a new extension". Three fields are filled in: "Extension Name: QuickLinksExtension", "Extension Display Name: Quick Links", "Output Path: C:\Dev\QuickLinksExtension". The Submit button is highlighted at the bottom.*

3.2 Generated Project Structure

The scaffold generates the following file and folder structure:

```
QuickLinksExtension/ │   Directory.Build.props             ← Central MSBuild
properties (TFM, SDK version) │   Directory.Packages.props          ←
Central Package Management (NuGet versions) │   nuget.config
← NuGet source configuration │   QuickLinksExtension.sln           ← Visual
Studio solution file │ └──QuickLinksExtension/              ← Main project
folder     │   app.manifest                  ← Win32 application manifest
(DPI awareness, UTF-8)     │   Package.appxmanifest          ← MSIX package
manifest (COM server, app extension)     │   Program.cs                       ←
Entry point: starts the COM server host     │   QuickLinksExtension.cs
← IExtension implementation (has CLSID GUID)     │
QuickLinksExtension.csproj     ← Project file (SDK-style, references Toolkit)
│   QuickLinksExtensionCommandsProvider.cs  ← ICommandsProvider
implementation     │     ├──Assets/     │       StoreLogo.png             ←
Store logo (50x50)     │       Square150x150Logo.png      ← Medium tile icon
│       Square44x44Logo.png        ← Small tile / taskbar icon     │
(Additional icon sizes)     │     ├──Pages/     │
QuickLinksExtensionPage.cs ← Default ListPage shown when extension is invoked
│     └──Properties/         │   launchSettings.json        ← VS launch
profile for --com-server arg         └──PublishProfiles/
win-arm64.pubxml       ← Publish profile for ARM64 (Surface Pro X, etc.)
win-x64.pubxml         ← Publish profile for x64 (most PCs)
```

3.3 Key Files Explained

Program.cs — COM Server Entry Point

The application entry point starts a **COM server host** that listens for COM activation requests from the Command Palette host. When Command Palette needs your extension, it activates your registered CLSID, which launches this executable with the `--com-server` argument. `Program.cs` detects this argument and starts the COM host loop. Without this, the process would exit immediately instead of waiting for COM calls.

QuickLinksExtension.cs — IExtension Root

The central class that implements the `IExtension` interface. Notice the `[Guid("...")]` attribute on the class — this is the **CLSID** used for COM registration. It must match the `Class Id` value in `Package.appxmanifest` exactly.

> ⚠️ **Critical Warning**
>
> **Never change the GUID after you have published your extension.** The CLSID is how the host identifies and activates your extension across updates. Changing it will break existing installations and prevent Command Palette from loading your extension. If you need a separate test identity, create a new project with a new GUID and a different package name.

QuickLinksExtensionCommandsProvider.cs — ICommandsProvider

Implements `ICommandsProvider`. This class is what the host queries (via `IExtension.GetProvider<ICommandsProvider>()`) to retrieve the list of top-level commands shown to the user. The `TopLevelCommands()` method returns an array of `ICommandItem` objects.

Pages/QuickLinksExtensionPage.cs — Default ListPage

The first page users see when they invoke your top-level command. Generated as a `ListPage` subclass (from the Toolkit). You will replace the placeholder items with real QuickLinks data in Part 4.

Package.appxmanifest — MSIX Manifest

Declares the COM server registration (`windows.comServer`) and the app extension (`com.microsoft.commandpalette`) that Command Palette uses for discovery. You generally do not need to hand-edit this file; the scaffold populates it correctly.

3.4 Alternative: Manual Project Setup via Visual Studio

If you cannot use the Command Palette scaffolding command (e.g., Command Palette is not yet installed, or you are setting up a CI environment), you can create the project manually.

10. In Visual Studio: **File** → **New** → **Project**. Search for **Blank App, Packaged (WinUI 3 in Desktop)** and select the C# version. Click **Next**.
11. Set the project name to QuickLinksExtension, choose your output path, and click **Create**.
12. Add NuGet packages via: **Tools** → **NuGet Package Manager** → **Manage NuGet Packages for Solution**. Search for and install:
    - Microsoft.CommandPalette.Extensions
    - Microsoft.CommandPalette.Extensions.Toolkit
13. Open Package.appxmanifest in the XML editor and add the following extension declarations inside the <Extensions> element. Replace {YOUR-GUID-HERE} with a freshly generated GUID (use **Tools** → **Create GUID** in Visual Studio):

```
<Extensions>   <!-- COM Server registration: tells Windows how to activate
the extension -->   <com:Extension Category="windows.comServer"
xmlns:com="http://schemas.microsoft.com/appx/manifest/com/windows10">
<com:ComServer>       <com:ExeServer Executable="QuickLinksExtension.exe"
Arguments="--com-server"                      DisplayName="QuickLinks
Extension">         <com:Class Id="{YOUR-GUID-HERE}" DisplayName="QuickLinks"
/>       </com:ExeServer>     </com:ComServer>   </com:Extension>    <!-- App
Extension: declares this as a Command Palette extension -->   <uap3:Extension
Category="windows.appExtension"
xmlns:uap3="http://schemas.microsoft.com/appx/manifest/uap/windows10/3">
<uap3:AppExtension Name="com.microsoft.commandpalette"
Id="QuickLinksExtension"                        DisplayName="Quick Links"
Description="Open configurable URLs from Command Palette"
```

```
PublicFolder="Public">     </uap3:AppExtension>   </uap3:Extension>
</Extensions>
```

> 💡 **Tip**
>
> Using the built-in "Create a new extension" scaffolding (Section 3.1) is strongly recommended over manual setup. The scaffolded project includes correctly configured `Directory.Build.props`, `Directory.Packages.props`, publish profiles, and `launchSettings.json` that would all need to be created manually otherwise.

Part 4 — Writing C# Commands and Pages

4.1 Anatomy of an IExtension Implementation

The `IExtension` interface is the root contract between your extension and the Command Palette host. Below is the complete, annotated implementation for `QuickLinksExtension.cs`:

```
using System; using System.Runtime.InteropServices; using
Microsoft.CommandPalette.Extensions;  namespace QuickLinksExtension;  ///
<summary> /// Root extension class. The GUID below is the CLSID registered in
/// Package.appxmanifest. DO NOT change this after first deployment. ///
</summary> [Guid("A1B2C3D4-E5F6-7890-ABCD-EF1234567890")]  // <-- Must match
manifest Class Id public sealed class QuickLinksExtension : IExtension,
IDisposable {     // The single CommandsProvider instance for this extension.
// Lazy-initialized so it is only created when the host requests it.
private readonly QuickLinksExtensionCommandsProvider _provider =
new();      /// <summary>     /// Called by the Command Palette host to
retrieve a capability provider.     /// Currently the only capability is
ICommandsProvider.     /// Return null for any type T that this extension
does not support.     /// </summary>     public object?
GetProvider(ProviderType providerType)     {         return providerType
switch         {             ProviderType.Commands => _provider,
_                   => null,         };     }      /// <summary>     ///
Generic overload — used by the SDK internally.     /// Delegates to the non-
generic GetProvider above.     /// </summary>     public T? GetProvider<T>()
where T : class     {         if (typeof(T) == typeof(ICommandsProvider))
return _provider as T;         return null;     }      /// <summary>     ///
```

```
Release managed resources when the host is done with this extension.     ///
</summary>     public void Dispose()     {         // Dispose the provider
and any held resources here.         // For simple extensions this can remain
empty.     } }
```

4.2 Implementing ICommandsProvider

The `ICommandsProvider` is what the host queries to get the list of commands shown at the top level when a user types your extension's name. Below is the full implementation:

```
using System; using Microsoft.CommandPalette.Extensions; using
Microsoft.CommandPalette.Extensions.Toolkit;  namespace QuickLinksExtension;
/// <summary> /// Provides the top-level commands for the QuickLinks
extension. /// The Command Palette host calls TopLevelCommands() each time it
/// needs to refresh the list of available commands for this extension. ///
</summary> internal sealed class QuickLinksExtensionCommandsProvider :
ICommandsProvider {     // Display name shown in the Command Palette header /
search results.     public string DisplayName => "Quick Links";      // Icon
displayed next to the extension name in the palette.     // Using a Segoe
Fluent icon (globe/world) — code point E909.     public IconInfo Icon =>
new("\uE909");      // The single top-level command: opens the QuickLinks
ListPage.     private readonly ICommandItem[] _commands;      public
QuickLinksExtensionCommandsProvider()     {         // Create the page that
will be shown when the user selects this command.         var page = new
QuickLinksExtensionPage();          // Wrap it in a ListItem so it appears as
a selectable command.         _commands = [             new ListItem(page)
{                 Title    = "Open Quick Links",                 Subtitle =
"Browse and launch your saved URLs",                 Icon     = new("\
uE71B"),  // Segoe Fluent: Link icon             }         ];     }      ///
<summary>     /// Returns the array of top-level commands.     /// Called by
the host whenever it needs to display or refresh the list.     /// Keep this
method fast — it may be called frequently.     /// </summary>     public
ICommandItem[] TopLevelCommands() => _commands; }
```

4.3 Building a List Page

The `ListPage` base class from the Toolkit handles all the rendering, keyboard navigation, and search filtering. You only need to implement `GetItems()` to return your data as an array of `IListItem` objects.

```
using System; using System.Collections.Generic; using System.Diagnostics;
using Microsoft.CommandPalette.Extensions; using
Microsoft.CommandPalette.Extensions.Toolkit;  namespace
QuickLinksExtension.Pages;  /// <summary> /// Represents a single Quick Link
(a named URL). /// </summary> internal record QuickLink(string Name, string
Url);  /// <summary> /// The main list page for the QuickLinks extension. ///
Inherits from the Toolkit's ListPage base class to reduce boilerplate. ///
```

```
</summary> internal sealed class QuickLinksExtensionPage : ListPage {     //
Sample data — in Part 4.5 we replace this with persisted settings.
private readonly List<QuickLink> _links =     [         new("Azure Portal",
"https://portal.azure.com"),         new("GitHub",
"https://github.com"),         new("Microsoft Learn",
"https://learn.microsoft.com"),         new("Jira Board",
"https://your-org.atlassian.net/jira"),         new("Team Wiki",
"https://your-org.sharepoint.com/sites/wiki"),         new("Stack Overflow",
"https://stackoverflow.com"),     ];      public QuickLinksExtensionPage()
{         Icon    = IconHelpers.FromRelativePath("Assets\\StoreLogo.png");
Title   = "Quick Links";         Name    = "Open Quick Links";         //
Enable the built-in search/filter bar at the top of the page.
ShowDetails = false;     }      /// <summary>     /// Returns the list of
items to display. Called each time the page is shown     /// or refreshed.
Keep this method fast — avoid heavy I/O here.     /// </summary>     public
override IListItem[] GetItems()     {         try         {             var
items = new IListItem[_links.Count];             for (int i = 0; i <
_links.Count; i++)             {                 var link = _links[i];
items[i] = new ListItem(new OpenLinkCommand(link.Url))                 {
Title    = link.Name,                     Subtitle = link.Url,
Icon     = new("\uE71B"),  // Link icon                 };             }
return items;         }         catch (Exception ex)         {             //
Never let an exception propagate to the host uncaught.             // Return
a single error item so the user knows something went wrong.
return [                 new ListItem(new NoOpCommand())                 {
Title    = "Error loading links",                     Subtitle = ex.Message,
Icon     = new("\uE783"),  // Error/warning icon                 }
];         }     } }  /// <summary> /// Command that opens a URL in the
user's default browser. /// Extends InvokableCommand from the Toolkit. ///
</summary> internal sealed class OpenLinkCommand : InvokableCommand
{     private readonly string _url;      public OpenLinkCommand(string url)
{         _url = url;         Name = "Open in Browser";         Icon = new("\
uE8A7");  // Segoe: Browser/Globe     }      public override ICommandResult
Invoke()     {         Process.Start(new ProcessStartInfo(_url)
{ UseShellExecute = true });         // Return a result that hides the
palette after the action.         return CommandResult.Hide();     } }
```

> 💡 **Tip**
>
> Use the Toolkit's built-in `OpenUrlCommand` helper class instead of writing your own `OpenLinkCommand` — it already handles `Process.Start` and `CommandResult.Hide()` for you. The custom implementation above is shown for educational purposes.

4.4 Adding a Command Result with Feedback

The `ICommandResult` returned from `Invoke()` controls what Command Palette does after your command runs. The SDK provides several result types:

Result	Behavior
`CommandResult.Hide()`	Closes/hides the Command Palette overlay.
`CommandResult.KeepOpen()`	Keeps the palette open on the current page.
`CommandResult.GoToPage(page)`	Navigates the palette to a different page.
`CommandResult.Dismiss()`	Navigates back to the previous page (like pressing Escape).
`CommandResult.ShowToast(message)`	Shows a short toast notification, then returns to the palette.

To show a toast notification after opening a link:

```
public override ICommandResult Invoke() {     Process.Start(new
ProcessStartInfo(_url) { UseShellExecute = true });     // Show a brief
confirmation toast, then hide the palette.     return
CommandResult.ShowToast($"Opened: {_url}"); }
```

To navigate to a new page (e.g., an "Add Link" form) from within a list item command:

```
public override ICommandResult Invoke() {     // Navigate the palette to an
AddLinkFormPage instead of hiding.     return CommandResult.GoToPage(new
AddLinkFormPage()); }
```

4.5 Working with Form Pages

A `FormPage` lets users enter data directly in the palette. Below is an example of an "Add Quick Link" form page with Name and URL text fields:

```
using Microsoft.CommandPalette.Extensions; using
Microsoft.CommandPalette.Extensions.Toolkit;  namespace
QuickLinksExtension.Pages;  internal sealed class AddLinkFormPage : FormPage
{     public AddLinkFormPage()     {         Icon  = new("\uE710");  //
Add/Plus icon         Title = "Add a New Quick Link";         Name  = "Add
Quick Link";     }      public override IForm[] GetForms()
{         return [             new AddLinkForm()         ];     } }  internal
sealed class AddLinkForm : Form {     public AddLinkForm()     {         //
Define the JSON schema for the form fields.         // The Toolkit interprets
this schema to render form controls.         DataJson = """         {
"type": "object",             "properties": {                 "name": {
"type": "string",                     "title": "Link Name",
"description": "A short, descriptive name (e.g. 'Azure Portal')"
},                 "url": {                     "type": "string",
"title": "URL",                     "description": "The full URL to open
```

```
(e.g. 'https://portal.azure.com')"                }            },
"required": ["name", "url"]         }         """;          TemplateJson =
"""         {             "type": "AdaptiveCard",             "version":
"1.5",             "body": [                 { "type": "Input.Text", "id":
"name", "label": "Link Name",                   "placeholder": "Azure Portal"
},                 { "type": "Input.Text", "id": "url",  "label": "URL",
"placeholder": "https://portal.azure.com" }             ],
"actions": [                 { "type": "Action.Submit", "title": "Add Link" }
]         }         """;     }      // Called when the user clicks "Add
Link".     public override ICommandResult SubmitForm(string payload)     {
// payload is a JSON string: { "name": "...", "url": "..." }         // Parse
and save using your persistence layer (see Section 8.3).         var data =
System.Text.Json.JsonDocument.Parse(payload).RootElement;         string name
= data.GetProperty("name").GetString() ?? string.Empty;         string url  =
data.GetProperty("url").GetString()  ?? string.Empty;          // TODO:
Persist the new link via IExtensionSettings or LocalSettings.          return
CommandResult.ShowToast($"Added '{name}' successfully!");     } }
```

4.6 Displaying Markdown Content

A `MarkdownPage` is the easiest way to render rich text inside the palette. Below is a short "Help" page for the QuickLinks extension:

```
using Microsoft.CommandPalette.Extensions.Toolkit;  namespace
QuickLinksExtension.Pages;  internal sealed class QuickLinksHelpPage :
MarkdownPage {     public QuickLinksHelpPage()     {         Icon  = new("\
uE897");  // Help/Question mark icon         Title = "Quick Links — Help";
Name  = "Help";     }      public override string GetMarkdownContent() => """
# Quick Links — Help      **Quick Links** lets you open your favourite URLs
directly from     Command Palette — no browser address bar required.      ##
How to Use      - **Open the list** — Invoke Command Palette and type "Quick
Links".     - **Launch a link** — Select any link and press **Enter**.     -
**Add a link** — Select "Add Quick Link" to open the input form.     -
**Remove a link** — Right-click a link item and choose "Delete".      ## Tips
- Links are stored in your local app settings and persist across sessions.
- You can assign Segoe Fluent icons to individual links for visual clarity.
- Report issues at
[github.com/your-org/QuickLinksExtension](https://github.com).      ---
Version 1.0.0 · Built with the Command Palette Extension SDK     """; }
```

4.7 Setting Extension Icons

Icons appear next to your extension name in the palette, on individual list items, and in the Command Palette Settings panel. Follow these steps to add and reference icons correctly:

14. Place your PNG icon files in the `Assets\` folder of your project.

15. In Visual Studio's **Solution Explorer**, right-click each icon file → **Properties** → set **Build Action** to `Content` and **Copy to Output Directory** to `Copy if newer`.
16. Reference icons in `Package.appxmanifest` under the **Visual Assets** tab — or in the XML under `<Applications>` → `<Application>` → `<VisualElements>`.
17. In code, create `IconInfo` objects using `IconHelpers.FromRelativePath()`:

```
// From a file in the Assets folder: Icon =
IconHelpers.FromRelativePath("Assets\\StoreLogo.png");  // From a Segoe
Fluent Icons glyph (recommended for small icons): Icon = new IconInfo("\
uE71B");   // Link glyph  // From an emoji: Icon = new IconInfo("🔗");
```

Recommended icon sizes (PNG):

Size	Usage
16 × 16 px	Inline list item icons (small)
32 × 32 px	Standard palette icons
48 × 48 px	Medium tile / command header
256 × 256 px	Store listing, Gallery card, high-DPI displays

[Screenshot: Solution Explorer — Assets Folder with Icon Files]

Visual Studio Solution Explorer is open showing the QuickLinksExtension project tree. The "Assets" folder is expanded, revealing icon files: StoreLogo.png, Square44x44Logo.png, Square150x150Logo.png, and a custom QuickLinksIcon.png. Each file shows "Content" as its Build Action in the Properties panel on the right.

4.8 Best Practices for Command Development

- **Keep `GetItems()` fast.** The host may call it frequently. Load data asynchronously where possible, or pre-cache it in the constructor and refresh on explicit user action.
- **Cache results.** If fetching data from a network or disk, cache the result in a field and invalidate on user refresh — not on every keystroke.
- **Handle all exceptions.** Wrap your `GetItems()` body in a `try/catch` and return a meaningful error `ListItem` rather than letting exceptions propagate to the host.
- **Use Toolkit base classes.** `ListPage`, `MarkdownPage`, `InvokableCommand`, `OpenUrlCommand`, and `CopyTextCommand` drastically reduce boilerplate. Prefer them over implementing interfaces from scratch.
- **Keep CLSIDs stable.** Never change the `[Guid]` attribute on your `IExtension` class after first deployment. See Section 8.4.
- **Follow naming conventions.** Use the pattern *ExtensionName*`CommandsProvider`, *ExtensionName*`Page`, *ExtensionName*`Extension` for clarity and consistency.
- **Test both architectures.** Build and deploy using both `win-x64` and `win-arm64` publish profiles before releasing. ARM64 Windows devices (e.g., Snapdragon-based Surface) are increasingly common.
- **Avoid blocking the COM host thread.** If `Invoke()` performs I/O, use `Task.Run()` and return a `KeepOpen` result immediately while work completes in the background.

Part 5 — Testing the Extension Locally

5.1 Deploying from Visual Studio

18. Open `QuickLinksExtension.sln` in Visual Studio.
19. Set the build configuration to **Debug | x64** using the dropdowns in the toolbar.
20. In **Solution Explorer**, right-click the `QuickLinksExtension` project node and select **Deploy**.
    Alternatively, use the menu: **Build → Deploy Solution**.
21. Visual Studio builds the project and registers the MSIX package with Windows using Developer Mode (no Store required). Watch the **Output** window for progress.
22. A successful deploy shows: *"Deploy: 1 succeeded, 0 failed"* in the Status Bar.

[Screenshot: Visual Studio — Solution Explorer Deploy Context Menu]

Visual Studio Solution Explorer is shown with the "QuickLinksExtension" project node right-clicked. The context menu is open and the "Deploy" option is highlighted in blue. The build configuration dropdown in the toolbar shows "Debug | x64".

📝 **Note**

The first deploy takes longer because Windows must install and register the MSIX package. Subsequent deploys update the existing package registration and are significantly faster.

5.2 Launching and Testing in Command Palette

23. Open Command Palette with **Win+Alt+Space**.
24. In the search bar, type **Quick Links** (the extension's Display Name).
25. The top-level command "Open Quick Links" should appear in the results list.
26. Press **Enter** to open the `ListPage`. Verify that your configured links appear.
27. Click or press Enter on any link and confirm that the URL opens in your default browser.

> **[Screenshot: Command Palette — "Quick Links" Extension Results]**
>
> *The Command Palette overlay is open on the Windows desktop. The search bar contains "Quick Links". Below it, a list shows the "Open Quick Links" command with a link icon. A subtitle reads "Browse and launch your saved URLs". The extension name badge shows "Quick Links".*

5.3 Attaching the Debugger

Because the extension runs in its own process, standard F5 debugging does not automatically attach. Use the following approach:

28. Deploy the Debug build (Section 5.1). Do *not* start with F5 — use **Debug → Start Without Debugging (Ctrl+F5)** to deploy and register without attaching.
29. Open Command Palette and invoke your extension. This causes Windows to launch `QuickLinksExtension.exe` as a COM server.
30. Back in Visual Studio, open **Debug → Attach to Process (Ctrl+Alt+P)**.
31. In the **Attach to Process** dialog, search for `QuickLinksExtension.exe` in the process list.

32. Select it and click **Attach**. The debugger is now connected — set breakpoints as you normally would.

> 💡 **Tip**
>
> To automatically trigger a debugger break at startup, you can add `Debugger.Launch();` at the top of `Program.cs` (in the `#if DEBUG` block) for the initial debugging session. Remove it before releasing.

5.4 Using the Output and Debug Windows

- **View → Output** → Set "Show output from:" to **Debug**. This shows COM activation events, extension lifecycle messages, and any `Debug.WriteLine()` calls from your code.
- **View → Other Windows → Command Window** — Evaluate expressions and call methods during a debugging session.
- **Debug → Windows → Immediate Window (Ctrl+Alt+I)** — Another quick-evaluation window useful for inspecting variable states at breakpoints.

5.5 Redeploying After Changes

33. Stop the current debugging session (if any): **Debug → Stop Debugging (Shift+F5)**.
34. Make your code changes in Visual Studio.
35. Right-click the project in **Solution Explorer** → **Deploy** again.
36. Re-open Command Palette. If your changes do not appear immediately, wait a few seconds — Command Palette may hold a reference to the previous extension process briefly.

> 📝 **Note**
>
> Command Palette caches extension metadata between sessions. If your changes (particularly DisplayName, Icon, or commands) do not appear after redeployment, navigate to **PowerToys Settings → Command Palette → Extensions → Quick Links** and toggle the extension **Off**, then **On** again to force a reload.

5.6 Common Issues and Fixes

Issue	Likely Cause	Fix
Extension does not appear in Command Palette after deploy	Developer Mode not enabled, or extension toggle is off in PowerToys Settings	Enable Developer Mode: **Settings → System → For developers → Developer Mode → On**. Then verify the extension toggle in PowerToys Settings → Command Palette.
COM activation error / extension fails to load	GUID mismatch between `[Guid]` attribute in code and `Class Id` in `Package.appxmanifest`	Open both files side-by-side. Copy the exact GUID string from the manifest and paste it into the `[Guid("...")]` attribute. Redeploy.
Package deploy fails with "path not found" or long path error	Long Paths not enabled in the OS	Enable via Registry or PowerShell as shown in Section 2.4. Restart your machine.
Extension crashes immediately on opening	Unhandled exception in `GetItems()` or the constructor	Wrap all code paths in `try/catch`. Check the **View → Output → Debug** window for the exception message and stack trace.
Icons are missing or show as broken image	Incorrect asset path, or Build Action not set to `Content`	Verify the icon file exists in the `Assets\` folder. In Solution Explorer, right-click the file → Properties

Issue	Likely Cause	Fix
		→ set **Build Action: Content**, **Copy to Output Directory: Copy if newer**.
Deploy error: "Access denied" or "Certificate error"	Self-signed certificate expired or not trusted	Open `Package.appxmanifest` → **Packaging** tab → **Choose Certificate** → create a new self-signed certificate. Redeploy.

Part 6 — Packaging the Extension

6.1 Release Build Configuration

37. In the Visual Studio toolbar, change the configuration from **Debug** to **Release**. Keep the platform as **x64** (or switch to **arm64** for ARM builds).
38. Build the solution: **Build → Build Solution (Ctrl+Shift+B)**.
39. Verify that the **Error List** (**View → Error List**) shows 0 errors and 0 relevant warnings. Address any warnings related to nullability, obsolete APIs, or packaging before proceeding.

> 💡 **Tip**
>
> Enable compiler warnings-as-errors in Release builds by adding `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` to your `Directory.Build.props`. This prevents silent issues from shipping in your published extension.

6.2 Publishing with Publish Profiles

The scaffolded project includes two publish profiles for the two primary architectures. Using these profiles produces a self-contained output that includes the .NET runtime — users do not need a separate .NET installation.

40. In **Solution Explorer**, right-click the project → **Publish**.
41. In the Publish dialog, select the **win-x64** profile → click **Publish**.
42. Output is placed in: `bin\Release\net8.0-windows10.0.22621.0\win-x64\publish\`
43. Repeat for **win-arm64** if you want to support ARM64 devices.

For **WinGet distribution** (non-MSIX, standalone installer), modify the `.csproj` to remove MSIX packaging:

```
<!-- In QuickLinksExtension.csproj, inside the <PropertyGroup> -->  <!--
Remove this line: --> <!--
<PublishProfile>win-$(Platform).pubxml</PublishProfile> -->  <!-- Add this
instead to disable MSIX packaging for WinGet: -->
<WindowsPackageType>None</WindowsPackageType>  <!-- Also set SelfContained to
true for standalone distribution --> <SelfContained>true</SelfContained>
<RuntimeIdentifier>win-x64</RuntimeIdentifier>
```

6.3 Creating an MSIX Package (Microsoft Store)

For Store publishing, keep the default MSIX packaging and use Visual Studio's **Create App Packages** wizard:

44. Right-click the project in Solution Explorer → **Package and Publish → Create App Packages**.
45. On the "Select distribution method" screen, choose **Microsoft Store**.
46. Sign in with your **Microsoft Partner Center** account (create one at `partner.microsoft.com` if needed — free for individuals).
47. Select or reserve your app name in Partner Center.

48. Choose architectures: check both **x64** and **arm64** for the widest compatibility.
49. Click **Create**. Visual Studio builds the solution, signs the package, and produces an `.msixbundle` file in the `AppPackages\` folder.

> **[Screenshot: Visual Studio — Create App Packages Wizard]**
>
> *The "Create App Packages" wizard dialog is open in Visual Studio. The first screen shows two radio button options: "Microsoft Store" (selected, highlighted in blue) and "Sideloading". Below the options, the "Next" button is visible. The title bar reads "Create App Packages — QuickLinksExtension".*

6.4 Code Signing (Optional but Recommended for WinGet)

MSIX packages must be signed before deployment. For development, Visual Studio auto-generates a self-signed certificate. For production and WinGet distribution, a trusted certificate is required.

Self-Signed Certificate (Local Testing)

Navigate to: **Package.appxmanifest → Packaging tab → Choose Certificate → Create test certificate**. Visual Studio creates a `.pfx` file and adds it to the project.

Trusted CA Certificate (Production)

- Obtain a code-signing certificate from a trusted Certificate Authority (e.g., DigiCert, Sectigo).
- Import the certificate into your Windows Certificate Store.
- In the Packaging tab, click **Choose Certificate → Select from store** and pick your CA certificate.

- Alternatively, use `signtool.exe` from the Windows SDK after building:

```
signtool sign /fd SHA256 /a /tr http://timestamp.digicert.com /td SHA256 `
"AppPackages\QuickLinksExtension_1.0.0.0_x64.msix"
```

> 📝 **Note**
>
> For WinGet manifest submissions, you must provide the **SHA256 hash** of the installer file. Generate it with: `Get-FileHash .\QuickLinksExtension.msix -Algorithm SHA256` in PowerShell.

Part 7 — Publishing to the Gallery

7.1 Distribution Options Overview

Method	Auto-Updates	Requirements	Best For
Microsoft Store	Yes (automatic)	Partner Center account, MSIX package, Store certification	Broad consumer reach; recommended for polished, general-purpose extensions
WinGet	Yes (on `winget upgrade`)	WinGet YAML manifest + `windows-commandpalette-extension` tag, PR to winget-pkgs	Developer and power-user audience; fastest path to discoverability for dev tools
Gallery listing	Depends on source (Store or WinGet)	PR to microsoft/CmdPal-Extensions repo; JSON manifest + icon	In-app discoverability directly within Command Palette; complements Store/WinGet
GitHub Releases	No (manual)	GitHub repository;	Internal tools, team

Method	Auto-Updates	Requirements	Best For
/ direct link		self-signed or CA certificate	extensions, early previews, contributors only

7.2 Publishing to WinGet (Recommended for Developers)

Step 1 — Prepare the WinGet Manifest

Install **WingetCreate**, Microsoft's manifest generation tool:

```
winget install Microsoft.WingetCreate
```

Run the new-manifest wizard, pointing to your published installer URL:

```
wingetcreate new `
https://github.com/your-org/QuickLinksExtension/releases/download/v1.0.0/
QuickLinksExtension.msix
```

The wizard prompts for:

- **PackageIdentifier** — e.g., `YourName.QuickLinksExtension`
- **PackageVersion** — e.g., `1.0.0`
- **InstallerUrl** — your GitHub release download URL
- **SHA256** — auto-computed from the URL by WingetCreate

WingetCreate outputs three YAML files: `.installer.yaml`, `.locale.en-US.yaml`, and `.yaml` (root).

Step 2 — Add the Required Tag

Open your `.locale.en-US.yaml` file and add the Command Palette discovery tag.

This tag is what allows Command Palette's Gallery to find your extension on WinGet:

```
# In YourName.QuickLinksExtension.locale.en-US.yaml: Tags:   - windows-
commandpalette-extension   - productivity   - url-launcher
```

Step 3 — Add WindowsAppRuntime Dependency

Command Palette extensions require the Windows App Runtime. Declare this dependency in your `.installer.yaml`:

```
# In YourName.QuickLinksExtension.installer.yaml: Dependencies:
PackageDependencies:     - PackageIdentifier: Microsoft.WindowsAppRuntime.1.7
MinimumVersion: 1.7.0
```

Step 4 — Submit to WinGet

50. Fork the WinGet packages repository: `https://github.com/microsoft/winget-pkgs`
51. Create the folder path: `manifests/y/YourName/QuickLinksExtension/1.0.0/`
52. Copy your three YAML files into that folder.
53. Commit and push your branch, then open a **Pull Request** to `microsoft/winget-pkgs`.
54. Automated CI validates the manifest. The WinGet team reviews and merges — typically within 1–7 business days.

> 💡 **Tip**
>
> Use `wingetcreate validate .\manifests\` to validate your YAML files locally before submitting the PR. This catches common formatting errors before CI runs.

7.3 Publishing to the Microsoft Store

55. Create a Partner Center account at partner.microsoft.com (free for individual developers).
56. Navigate to: **Partner Center → Windows & Xbox → Overview → Create a new app**. Reserve your app name (*Quick Links*).

57. Under **Submissions → Create your first submission**, upload the .msixbundle generated in Section 6.3.
58. Fill in the Store listing:
    - **Description:** "Open your favourite URLs instantly from Command Palette — no browser required."
    - **Category:** Productivity → Developer Tools
    - **Screenshots:** At least two screenshots of the extension in action (1366×768 minimum).
    - **Store logo:** 300×300 PNG (no transparency).
59. Set pricing to **Free** (Command Palette extensions are free by convention).
60. Click **Submit to the Store**. Microsoft certification typically takes 1–3 business days.

> **[Screenshot: Microsoft Partner Center — New Submission Form]**
>
> *The Microsoft Partner Center web interface is shown. The "New submission" page for "Quick Links" is open. Fields visible include "Description", "Category" (set to "Developer Tools"), and a "Packages" section with "QuickLinksExtension_1.0.0.0_x64_arm64.msixbundle" listed as uploaded. A "Submit to the Store" button is in the upper-right corner.*

7.4 Listing in the Command Palette Extension Gallery

The **Command Palette Extension Gallery** (`microsoft/CmdPal-Extensions` on GitHub) is the in-app discovery surface — users can browse and install extensions

directly from Command Palette without leaving the app. Getting listed here dramatically increases visibility.

61. Fork the repository: `https://github.com/microsoft/CmdPal-Extensions`
62. Create a new folder: `extensions/YourName.QuickLinksExtension/`
63. Create an `extension.json` file in that folder with the following structure:

```
{   "id": "YourName.QuickLinksExtension",   "name": "Quick Links",
"description": "Open configurable URLs instantly from Command Palette.",
"publisher": "Your Name",   "icon": "icon.png",   "installSource": "winget",
"wingetPackageId": "YourName.QuickLinksExtension",   "tags": ["productivity",
"url-launcher", "browser"],   "supportUrl":
"https://github.com/your-org/QuickLinksExtension/issues",   "repositoryUrl":
"https://github.com/your-org/QuickLinksExtension" }
```

64. Add a **256 × 256 PNG icon** named `icon.png` to the same folder. This icon appears on the Gallery card in Command Palette.
65. Commit both files, push your branch, and open a **Pull Request** to `microsoft/CmdPal-Extensions`.
66. Automated CI validates the JSON and icon. The Command Palette team reviews the PR.
67. Once merged, your extension appears in the Command Palette Gallery — users can find and install it with a single click, directly from the palette.

> ⚠️ **Requirement**
>
> Your extension must be publicly available via WinGet or the Microsoft Store **before** submitting a Gallery PR. The Gallery entry links to your WinGet package ID or Store product ID — there must be a live, installable package for the link to work.

7.5 Keeping Your Extension Updated

- Increment the version in `Package.appxmanifest` (`<Identity Version="1.1.0.0" />`) on each release.

- For **WinGet**, run `wingetcreate update` with the new version and installer URL, then submit a new PR:

```
wingetcreate update YourName.QuickLinksExtension `     --version 1.1.0 `
--urls https://github.com/your-org/QuickLinksExtension/releases/download/
v1.1.0/QuickLinksExtension.msix
```

- For the **Microsoft Store**, create a new submission in Partner Center, upload the new `.msixbundle`, and submit. Store updates are pushed to existing users automatically.
- The **Gallery JSON** does not need to be updated for version bumps — it points to the WinGet package ID, and WinGet handles version resolution automatically.
- Keep a **CHANGELOG.md** in your repository so users know what changed in each release.

Part 8 — Best Practices

&

Advanced Topics

8.1 Performance Best Practices

- **Never block the COM host thread.** All page methods (`GetItems()`, `GetMarkdownContent()`, `GetForms()`) are called synchronously by the host. If your data requires I/O or network access, pre-load it in a background task and cache the result. Return the cached data synchronously from `GetItems()`.
- **Defer expensive work until the first open.** Don't load data in the constructor — lazy-load it on the first `GetItems()` call. Use a `bool _initialized` flag and a `Task _initTask` to control this.

- **Implement incremental search client-side.** Instead of making a network call for each search query, return all items from `GetItems()` and let the Toolkit's built-in filtering handle the search. Only use server-side search for very large datasets (>1,000 items).
- **Minimize allocations in hot paths.** `GetItems()` may be called frequently. Reuse `IListItem` instances where possible instead of recreating them on every call.

8.2 Accessibility

- **Always provide Title and Subtitle** on every `IListItem`. Screen readers like Narrator announce these aloud. A missing title results in an empty announcement, which is disorienting for blind users.
- **Provide tooltips on form fields.** Set the `description` property in your form JSON schema — it is rendered as a tooltip and read by Narrator when the field is focused.
- **Test with Narrator** (**Win+Ctrl+Enter** to toggle). Navigate through your extension entirely by keyboard and verify that every element is announced correctly and reachable.
- **Avoid emoji-only icons** as the sole identifier for list items. Always pair icons with a meaningful `Title` so that users relying on screen readers get the full context.
- **Respect high-contrast mode.** Use Segoe Fluent icon glyphs instead of raster images where possible — glyphs automatically adapt to high-contrast themes.

8.3 Handling Settings and Persistence

Extensions need to store user preferences (e.g., the list of Quick Links) persistently across sessions. There are two recommended approaches:

Option A — IExtensionSettings API (Recommended)

The Command Palette SDK provides a built-in settings API that integrates with the PowerToys Settings UI. Users can view and edit your extension's settings directly in **PowerToys Settings → Command Palette → Extensions → Quick Links → Settings**.

```
// Register a setting in the extension class constructor or Init method: var
settings = ExtensionSettings.Create(     new SettingDefinition<string>(
key:          "links_json",         defaultValue: "[]",         displayName:
"Saved Links",         description:  "JSON array of {name, url}
objects"     ) );  // Read a setting value: string json =
settings.Get<string>("links_json") ?? "[]";  // Write a setting value:
settings.Set("links_json",
System.Text.Json.JsonSerializer.Serialize(_links));
```

Option B — ApplicationData.Current.LocalSettings

For simple key-value storage not exposed in the PowerToys Settings UI:

```
using Windows.Storage;  // Write:
ApplicationData.Current.LocalSettings.Values["links_json"] =
System.Text.Json.JsonSerializer.Serialize(_links);  // Read: string json =
ApplicationData.Current.LocalSettings                .Values["links_json"]
as string ?? "[]";
```

> ⚠️ **Warning**
>
> Never hard-code file paths like `C:\Users\Thet\AppData\`. Always use `Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData)` or `ApplicationData.Current.LocalFolder.Path` to resolve user-specific directories at runtime.

8.4 Extension Versioning and GUIDs

- **CLSID stability is non-negotiable.** The `[Guid("...")]` on your `IExtension` class is the permanent identity of your extension from the moment it is first

installed by any user. Changing it severs the COM registration link and breaks all existing installations.

- **Use semantic versioning** in `Package.appxmanifest`: increment `MAJOR.MINOR.PATCH.BUILD` (e.g., `1.0.0.0` → `1.1.0.0`) on each release.
- **Separate test package identity.** If you need a test build with a different CLSID (e.g., to install alongside the production build), create a separate project with a different `PackageIdentityName` in the manifest (e.g., `YourName.QuickLinksExtension.Dev`) and a new GUID.
- **Keep a GUID registry.** Document all GUIDs used by your extension (including any historical ones) in a `GUIDS.md` file in your repository. This prevents accidental reuse.

8.5 Useful Resources

Resource	Description	URL
Microsoft Learn — Command Palette Extension Hub	Official Microsoft documentation: architecture, API reference, tutorials, and publishing guides.	learn.microsoft.com/windows/powertoys/command-palette
GitHub: microsoft/CmdPal-Extensions	The community extension Gallery repository. Browse existing extensions for reference and submit your own.	github.com/microsoft/CmdPal-Extensions
GitHub: microsoft/PowerToys	PowerToys open-source repository. Includes Command Palette source code and official sample extensions.	github.com/microsoft/PowerToys
NuGet: Microsoft.CommandPalette.Extensions	Core SDK package. Check the "Versions" tab for the latest stable release.	nuget.org/packages/Microsoft.CommandPalette.Extensions
NuGet: Microsoft.CommandPalette.Extensions.Toolki	Helper utilities and base classes. Always use the same version as the core SDK package.	nuget.org/packages/Microsoft.CommandPalette.Extensions.Toolkit

Resource	Description	URL
t		
WinGet Package Manager	Browse, search, and install WinGet packages. Useful for verifying your manifest after it is merged.	winget.run
Microsoft Partner Center	The Store publishing portal. Register, submit, and manage your app certifications here.	partner.microsoft.com
Segoe Fluent Icons Reference	Browse all Segoe Fluent icon glyphs and their Unicode code points. Essential for choosing extension icons.	learn.microsoft.com/windows/apps/design/style/segoe-fluent-icons-font

---

> Appendix A — Full QuickLinksExtension Code Reference

Program.cs — COM Host Entry Point

```
using Microsoft.CommandPalette.Extensions; using
Microsoft.Windows.AppLifecycle;  namespace QuickLinksExtension;  internal
static class Program {     [STAThread]     static void Main(string[] args)
{         // If the process was activated as a COM server, start the COM host
loop.         // Command Palette launches the extension exe with --com-
server.         if (args.Contains("--com-server"))         {             //
Register the extension class as a COM server.             // The Guid here
must match [Guid] on QuickLinksExtension class             // AND the Class
Id in Package.appxmanifest.             using var server = new
MarshalingExtensionServer();             server.Run<QuickLinksExtension>(
new Guid("A1B2C3D4-E5F6-7890-ABCD-
EF1234567890")             );             // The Run() call blocks until the
host releases all references.             return;         }          // If
launched normally (e.g., by clicking the exe),         // show a friendly
message and exit — this is not a standalone app.
Console.WriteLine("QuickLinksExtension is a Command Palette extension.");
Console.WriteLine("It is managed by PowerToys and cannot be run directly.");
} }
```

QuickLinksExtension.cs — IExtension Root

```
using System; using System.Runtime.InteropServices; using
Microsoft.CommandPalette.Extensions;  namespace QuickLinksExtension;  ///
<summary> /// Root extension class. /// The GUID must match
Package.appxmanifest Class Id — never change after publishing. /// </summary>
```

```
[Guid("A1B2C3D4-E5F6-7890-ABCD-EF1234567890")] public sealed class
QuickLinksExtension : IExtension, IDisposable {     private readonly
QuickLinksExtensionCommandsProvider _provider = new();     private bool
_disposed;      public object? GetProvider(ProviderType providerType) =>
providerType switch     {         ProviderType.Commands => _provider,
_                     => null,     };      public T? GetProvider<T>() where T
: class     {         if (typeof(T) == typeof(ICommandsProvider)) return
_provider as T;           return null;     }      public void Dispose()     {
if (!_disposed)           {             // Dispose any resources held by the
provider.             _disposed = true;         }     } }
```

QuickLinksExtensionCommandsProvider.cs — ICommandsProvider

```
using Microsoft.CommandPalette.Extensions; using
Microsoft.CommandPalette.Extensions.Toolkit; using QuickLinksExtension.Pages;
namespace QuickLinksExtension;  internal sealed class
QuickLinksExtensionCommandsProvider : ICommandsProvider {     public string
DisplayName => "Quick Links";     public IconInfo Icon        => new("\
uE909");  // Globe / Link icon      private readonly ICommandItem[]
_commands;      public QuickLinksExtensionCommandsProvider()
{         var page = new QuickLinksExtensionPage();         _commands =
[             new ListItem(page)             {                 Title    =
"Open Quick Links",                 Subtitle = "Browse and launch your saved
URLs",                  Icon     = new("\uE71B"),             },
new ListItem(new QuickLinksHelpPage())             {                 Title
= "Quick Links Help",                 Subtitle = "View usage instructions",
Icon     = new("\uE897"),             },         ];     }      public
ICommandItem[] TopLevelCommands() => _commands; }
```

Pages/QuickLinksExtensionPage.cs — ListPage with URL Items

```
using System; using System.Collections.Generic; using System.Diagnostics;
using Microsoft.CommandPalette.Extensions; using
Microsoft.CommandPalette.Extensions.Toolkit;  namespace
QuickLinksExtension.Pages;  internal record QuickLink(string Name, string
Url);  internal sealed class QuickLinksExtensionPage : ListPage {     // In
production: load this from IExtensionSettings or LocalSettings.     private
readonly List<QuickLink> _links =     [         new("Azure Portal",
"https://portal.azure.com"),         new("GitHub",
"https://github.com"),         new("Microsoft Learn",
"https://learn.microsoft.com"),         new("Jira Board",      "https://your-
org.atlassian.net/jira"),         new("Team Wiki",       "https://your-
org.sharepoint.com/sites/wiki"),         new("Stack Overflow",
"https://stackoverflow.com"),     ];      public QuickLinksExtensionPage()
{         Icon  = IconHelpers.FromRelativePath("Assets\\StoreLogo.png");
Title = "Quick Links";         Name  = "Open Quick Links";     }      public
override IListItem[] GetItems()     {         try         {             var
items = new IListItem[_links.Count + 1];             for (int i = 0; i <
_links.Count; i++)             {                 var link = _links[i];
items[i] = new ListItem(new OpenLinkCommand(link.Url))                 {
Title    = link.Name,                     Subtitle = link.Url,
Icon     = new("\uE71B"),                 };             }             //
Append an "Add New Link" action at the bottom of the list.
items[^1] = new ListItem(new GoToPageCommand(new AddLinkFormPage()))
{                 Title    = "Add New Link...",                 Subtitle =
"Open the form to add a new Quick Link",                 Icon     = new("\
uE710"),  // Plus/Add icon             };             return items;         }
```

```
catch (Exception ex)         {             return [                 new
ListItem(new NoOpCommand())                 {                     Title    =
"Error loading Quick Links",                     Subtitle = ex.Message,
Icon     = new("\uE783"),                 }             ];         }     } }
internal sealed class OpenLinkCommand : InvokableCommand {     private
readonly string _url;      public OpenLinkCommand(string url)
{         _url = url;         Name = "Open in Browser";         Icon = new("\
uE8A7");     }      public override ICommandResult Invoke()
{         Process.Start(new ProcessStartInfo(_url) { UseShellExecute =
true });         return CommandResult.ShowToast($"Opened: {_url}");     } }
internal sealed class GoToPageCommand : InvokableCommand {     private
readonly IPage _page;      public GoToPageCommand(IPage page)
{         _page = page;         Name  = "Navigate";     }      public
override ICommandResult Invoke() => CommandResult.GoToPage(_page); }
```

> Appendix B — Glossary

Term	Definition
Command Palette	The next-generation Windows launcher included in Microsoft PowerToys (v0.90+). A fast, keyboard-driven, WinUI-based overlay that hosts extensions providing commands, search results, forms, and rich content without requiring separate application windows.
IExtension	The root interface that every Command Palette extension class must implement. The host calls `GetProvider<T>()` on it to discover the extension's capabilities (e.g., `ICommandsProvider`).
ICommandsProvider	An interface that exposes an array of `ICommandItem` objects — the top-level commands shown in the palette when a user searches for the extension. Returned by `IExtension.GetProvider<ICommandsProvider>()`.
CLSID	Class Identifier. A 128-bit GUID that uniquely identifies a COM class. In Command Palette extensions, the CLSID on the `IExtension` class must match the `Class Id` declared in `Package.appxmanifest`. Must never change after first deployment.

Term	Definition
COM	Component Object Model. Microsoft's binary interface standard for inter-process communication on Windows. Command Palette uses COM to activate and communicate with extension processes without requiring them to run in-process with the host.
WinUI	Windows UI Library (WinUI 3). The modern native UI framework for Windows apps built on the Windows App SDK. Command Palette's host shell is built with WinUI 3. Extension page types are rendered by the host's WinUI layer.
ListPage	A Command Palette SDK page type (from the Toolkit) that renders a scrollable, searchable list of `IListItem` rows. The most commonly used page type for navigation menus, search results, and launchers.
MSIX	Microsoft's modern app packaging format, replacing MSI and AppX. MSIX packages include the application binary, assets, dependencies, and a manifest. They support automatic updates, clean uninstallation, and are required for Microsoft Store distribution.
WinGet	The Windows Package Manager. A command-line tool (`winget`) and ecosystem that allows developers to publish apps as YAML manifests in the `microsoft/winget-pkgs` repository. Supports automatic updates and is the recommended distribution channel for developer-focused extensions.
SDK	Software Development Kit. In this guide, refers specifically to the Command Palette Extension SDK — the two NuGet packages (`Microsoft.CommandPalette.Extensions` and `Microsoft.CommandPalette.Extensions.Toolkit`) that provide the interfaces and base classes for building extensions.
Gallery	The in-app extension discovery surface built into Command Palette. Powered by the `microsoft/CmdPal-Extensions` GitHub repository. Users can browse, preview, and install extensions directly from

Term	Definition
	within Command Palette without visiting an external website.
OOP Extension	Out-of-Process Extension. An extension that runs in a separate process from the host. The Command Palette extension model is OOP — your extension's crash or hang does not affect the host or other extensions.
IListItem	An interface representing a single row in a `ListPage`. Contains `Title`, `Subtitle`, `Icon`, and an `IInvokableCommand` to execute when the row is selected.
CommandResult	The return value from an `IInvokableCommand.Invoke()` call. Controls what Command Palette does after the command runs — options include Hide, KeepOpen, GoToPage, ShowToast, and Dismiss.
